FIVE-STAGE IMPLEMENTATION PLAN

FROM JAVA MODEL TO RELIABLE FLIGHT BOOKING

Turn a tested academic Java domain model into a demonstrable, concurrency-safe flight reservation backend.

THE ENGINEERING PROBLEM

Two people request the same flight seat at the same moment. The system must accept exactly one valid reservation.

Request A: 12A — **Request B: 12A** — 1 success, 1 conflict

01 DEMO — 02 SEATS — 03 DATABASE — 04 API — 05 SHIP

WHAT THIS PLAN OPTIMIZES FOR

Correctness first. Clean integration second. Portfolio evidence third. No unnecessary full-stack rewrite.

01 / THE STARTING POINT

Keep the core. Fix the missing proof.

The current repository already shows useful object-oriented modeling and tests. The next step is a focused extension, not a rewrite.

WHAT YOU ALREADY HAVE

- Java 21, Gradle and JUnit 5
- Flight, passenger and reservation models
- State and Strategy patterns
- Existing unit tests and README

WHAT THE PROJECT LACKS

- Seat-level booking and inventory
- Atomic concurrent reservation control
- Persistent reservations
- HTTP API and inspectable requests

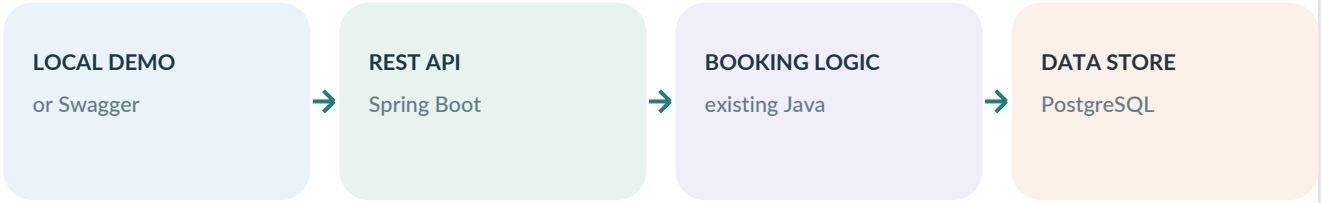
WHAT TO REPAIR

- Non-thread-safe static reservation counter
- Payment state changes before debit succeeds
- Public state setter bypasses lifecycle rules
- Booking workflows are not yet end-to-end

WHAT TO PRESERVE

- Existing domain names and behavior
- Pricing rules and State transitions
- Tests as regression protection
- Focus on backend engineering depth

TARGET REQUEST FLOW



Scope: One reliable local booking flow. No production hosting, authentication system, or real payment processing required.

Source distinction: Srikanth proposed a visible demo and concurrency challenge. Spring Boot and PostgreSQL are recommended implementation choices for the extension.

02 / BUILD SEQUENCE

Weeks 1–2 | Prove the booking rules

Treat these as outcome-based sprints. If exam preparation reduces available time, preserve the order rather than rushing the milestones.

WEEK 01

Make the existing system runnable

1. From FlightManagement/, run `./gradlew clean test` and document the result.
2. Create a scripted demo: flight → reservation → simulated payment → confirmation.
3. Add one invalid transition and a readable explanation of the output.
4. Update README with setup, test command, demo command, and limitations.

DONE WHEN: A stranger can run and understand one complete local flow.

WEEK 02

Introduce seat-level concurrency safety

1. Add a seat number and define the identity of a seat as (flightId, seatNumber).
2. Build a BookingService that checks availability and creates a booking atomically in one JVM.
3. Replace the factory's shared `compteur++` with safe unique identifiers.
4. Use CountDownLatch to launch 20 competing tasks for seat 12A.
5. Test release on cancellation and failures before adding new frameworks.

DONE WHEN: 20 contenders → exactly 1 success, across 100 repeated runs.

Decision gate: do not proceed to API work until the booking invariant is tested.

03 / BUILD SEQUENCE

Weeks 3–5 | Expose, persist, present

Progress only when the prior stage is demonstrable. This sequence keeps the strongest engineering story in the center.

WEEK 03

Persist the booking invariant

- Add PostgreSQL and a seat_assignments table with a unique flight/seat key.
- Create reservation + seat assignment in one transaction.
- Verify conflict handling, cancellation, and data after application restart.

PASS: two concurrent transactions cannot commit the same flight/seat.

WEEK 04

Add a small HTTP API

- Introduce Spring Boot with thin controllers and a service layer.
- Expose GET flights, GET seats, POST reservation, GET / DELETE reservation.
- Return clear 201 Created, 400 Bad Request, 404 Not Found, 409 Conflict responses.

PASS: API requests show both a booking success and a seat conflict.

WEEK 05

Polish the public portfolio evidence

- Expose endpoints through OpenAPI / Swagger UI (only after real HTTP endpoints exist).
- Add GitHub Actions to run tests and a clean-start README with sample requests.
- Capture local screenshots or a short GIF; add one architecture diagram.
- Explain the race condition, the safeguard, and the test result in plain English.

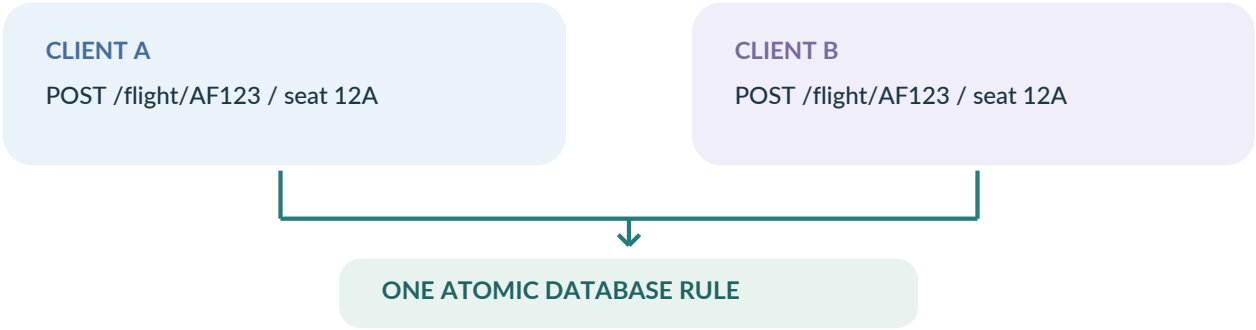
PASS: anyone can reproduce the demo locally using documented steps.

04 / THE CENTERPIECE

Two requests. One seat. One winner.

The most valuable portfolio claim is a correctness guarantee: no two active reservations can own the same flight seat.

CONCURRENT INPUT



THE MINIMAL UNIQUENESS CONSTRAINT

```
CREATE TABLE seat_assignments (
  flight_id      UUID NOT NULL,
  seat_number    VARCHAR(5) NOT NULL,
  reservation_id UUID NOT NULL UNIQUE,
  PRIMARY KEY (flight_id, seat_number)
);
```

Important: create the reservation and seat assignment in the same transaction. A seat conflict must roll back the operation. In-memory locks alone do not protect multiple application instances.

ACCEPTANCE TESTS

RACE-01	20 requests, same seat	1 success; 19 conflicts
RACE-02	Different seats	Both succeed
LIFE-01	Cancel then rebook	Seat becomes available
DATA-01	Restart application	Reservation persists
RULE-01	Invalid state transition	Rejected predictably

05 / SHIP WITH EVIDENCE

Make the GitHub project easy to trust.

The final outcome is not a big web app. It is a small, correct system that a recruiter can inspect and you can defend in an interview.

README STORYBOARD

01 Problem + booking invariant

02 Quick-start commands

03 API examples + statuses

04 Architecture and trade-offs

05 Concurrency test evidence

06 Limitations + next steps

RELEASE CHECKLIST

- [ ] Tests pass from a clean checkout
- [ ] One success and 409 demo work
- [ ] No secrets or credentials committed
- [ ] Local setup is documented
- [ ] Architecture diagram is readable
- [ ] Claims match measured evidence
- [ ] Original team is credited clearly

90-SECOND RECRUITER DEMO

01 SHOW	02 TRIGGER	03 EXPLAIN
flight + seat	two simultaneous bookings	1 created + 1 conflict

CV BULLET / ONLY AFTER IMPLEMENTATION

"Extended an academic Java reservation system into a REST backend with transaction-safe seat assignment; validated duplicate-booking prevention under concurrent requests."

NOT NOW

Real payments · Complex frontend · Microservices · Kubernetes · Production hosting